

Plano de Aprendizado: Go (Golang) — Do Zero ao Primeiro Sistema

Projeto que evolui com você: um gerenciador de tarefas chamado `taskctl`. Começa como um script que imprime no terminal. Termina como uma API REST conectada a PostgreSQL.

Fase 1 — Sintaxe Completa do Go

Objetivo: sair do zero absoluto e dominar toda a sintaxe da linguagem antes de construir qualquer coisa. Ao final desta fase, você lê qualquer código Go e entende o que está acontecendo.

1.1 — Instalação e Ambiente

O que você aprende: como instalar Go, configurar o workspace, e entender o sistema de módulos.

Go não precisa de IDE pesada. Você instala o binário, configura o `GOPATH` (ou usa o padrão `~/go`), e já roda código.

Instalar Go (Linux)

```
wget https://go.dev/dl/go1.22.5.linux-amd64.tar.gz
```

```
sudo tar -C /usr/local -xzf go1.22.5.linux-amd64.tar.gz
```

```
export PATH=$PATH:/usr/local/go/bin
```

Verificar

```
go version
```

Conceito importante — Go Modules: Diferente de PHP (Composer) ou Python (pip + venv), Go usa módulos nativos. Todo projeto Go começa com `go mod init`.

```
mkdir taskctl && cd taskctl
```

```
go mod init github.com/aerlon/taskctl
```

Isso cria um `go.mod` — equivalente ao `composer.json` ou `requirements.txt`. Dependências são gerenciadas automaticamente.

Primeiro arquivo:

```
// main.go

package main

import "fmt"

func main() {

    fmt.Println("taskctl v0.1 — gerenciador de tarefas")

}

go run main.go
```

Pontos-chave:

- Todo executável Go precisa do `package main` e da função `main()`.
- `fmt` é o pacote de formatação (equivalente ao `echo/print`).
- `go run` compila e executa. `go build` gera um binário compilado.

1.2 — Variáveis, Tipos e Constantes

O que você aprende: Go é **estaticamente tipada** (como Java, diferente de PHP/Python). Você declara o tipo ou Go infere. Não existe variável sem tipo.

```
package main

import "fmt"

func main() {

    // Declaração explícita (tipo declarado)

    var nome string = "Aerlon"
```

```

var idade int = 28

// Declaração curta (tipo inferido — só funciona dentro de funções)

cidade := "Goiânia"

ativo := true

// Constantes (não mudam nunca)

const versao = "0.1.0"

// Tipos numéricos principais

var inteiro int = 42      // int genérico (32 ou 64 bits conforme o OS)

var decimal float64 = 3.14 // ponto flutuante (sempre use float64, não float32)

var byte_ byte = 255      // alias pra uint8

// Zero values — Go inicializa variáveis com valor zero se você não atribuir

var textoVazio string // "" (string vazia)

var numeroZero int    // 0

var falso bool        // false

fmt.Println(nome, idade, cidade, ativo, versao)

fmt.Println(inteiro, decimal, byte_)

fmt.Println(textoVazio, numeroZero, falso)

}

```

Comparação com PHP/Python: | Conceito | PHP | Python | Go | ---|---|---|---| | Declarar variável | `$x = 10;` | `x = 10` | `x := 10` ou `var x int = 10` | | Tipo | Dinâmico | Dinâmico | Estático (inferido ou explícito) | | Constante | `const X = 10;` | `X = 10` (convenção) | `const X = 10` (real, imutável) | | Null/None/Nil | `null` | `None` | `nil` (só pra ponteiros, slices, maps, interfaces) |

Exercício com taskctl:

```
package main

import "fmt"

func main() {

    // Variáveis do nosso gerenciador de tarefas

    nomeApp := "taskctl"

    versao := "0.1.0"

    totalTarefas := 0

    rodando := true

    fmt.Printf("%s v%s | Tarefas: %d | Rodando: %t\n",

        nomeApp, versao, totalTarefas, rodando)

}
```

Ponto importante — **Printf** vs **Println**:

- **Println** imprime e pula linha. Simples.
- **Printf** formata com verbos: **%s** (string), **%d** (int), **%t** (bool), **%f** (float), **%v** (qualquer tipo).

1.3 — Arrays, Slices e Maps

O que você aprende: as 3 estruturas de dados fundamentais do Go. Slices são o que você mais vai usar (equivalente a arrays dinâmicos de PHP/Python). Maps são equivalentes a dicionários/arrays associativos.

```
package main

import "fmt"

func main() {
```

```

// ARRAY — tamanho fixo (raramente usado diretamente)

var diasSemana [7]string

diasSemana[0] = "Segunda"

diasSemana[1] = "Terça"

fmt.Println(diasSemana) // [Segunda Terça  ]

// SLICE — tamanho dinâmico (o que você vai usar 99% do tempo)

tarefas := []string{"Estudar Go", "Fazer CRUD", "Conectar banco"}

fmt.Println(tarefas)    // [Estudar Go Fazer CRUD Conectar banco]

fmt.Println(len(tarefas)) // 3

// Adicionar elemento (equivalente a append/push)

tarefas = append(tarefas, "Deploy")

fmt.Println(tarefas) // [Estudar Go Fazer CRUD Conectar banco Deploy]

// Acessar por índice

fmt.Println(tarefas[0]) // Estudar Go

// Slice de slice (fatiamento — como Python)

fmt.Println(tarefas[1:3]) // [Fazer CRUD Conectar banco]

// MAP — chave:valor (equivalente a dict do Python, array associativo do PHP)

status := map[string]int{

    "pendente": 3,

    "feito":    1,

    "cancelado": 0,

```

```

    }

    fmt.Println(status["pendente"]) // 3

    // Adicionar nova chave

    status["em progresso"] = 2

    // Verificar se chave existe (idioma do Go — muito importante!)

    valor, existe := status["feito"]

    if existe {

        fmt.Printf("feito: %d\n", valor)

    }

    // Deletar chave

    delete(status, "cancelado")

    // Iterar sobre map

    for chave, valor := range status {

        fmt.Printf("%s: %d\n", chave, valor)

    }

}

```

Conceito crítico — Slices vs Arrays: Arrays em Go têm tamanho fixo e são raramente usados. Slices são referências a arrays por baixo dos panos, com tamanho dinâmico. Quando alguém diz "array" em Go, geralmente quer dizer slice.

Exercício com taskctl:

```

package main

import "fmt"

func main() {

```

```
// Lista de tarefas como slice de strings (por enquanto simples)

tarefas := []string{}

// Simulando adição de tarefas

tarefas = append(tarefas, "Aprender sintaxe Go")

tarefas = append(tarefas, "Construir CRUD")

tarefas = append(tarefas, "Conectar PostgreSQL")

// Contadores por status usando map

contadores := map[string]int{

    "total":  len(tarefas),

    "pendente": len(tarefas),

    "feito":  0,

}

fmt.Println("=== taskctl ===")

for i, tarefa := range tarefas {

    fmt.Printf("[%d] %s\n", i+1, tarefa)

}

fmt.Printf("\nTotal: %d | Pendentes: %d | Feitas: %d\n",

    contadores["total"], contadores["pendente"], contadores["feito"])

}
```

1.4 — Controle de Fluxo (if, for, switch)

O que você aprende: Go tem apenas **for** para loops (não existe **while** ou **do-while**). O **if** pode ter uma declaração curta antes da condição. O **switch** é mais poderoso que em outras linguagens.

```
package main

import "fmt"

func main() {

    // IF — sem parênteses na condição (obrigatório em Go)

    idade := 28

    if idade >= 18 {

        fmt.Println("Maior de idade")

    }

    // IF com declaração curta (idioma do Go — muito comum)

    // A variável 'err' só existe dentro do bloco if/else

    if x := 10 * 2; x > 15 {

        fmt.Println("x é maior que 15:", x)

    }

    // FOR — único loop do Go (substitui while, for, foreach)

    // For clássico

    for i := 0; i < 5; i++ {

        fmt.Println(i)

    }
```



```
// For como "while"

contador := 0

for contador < 3 {

    fmt.Println("contando:", contador)

    contador++

}

// For infinito (equivalente a while true)

// for {

//     fmt.Println("loop infinito")

//     break // sem break, roda pra sempre

// }

// For range (equivalente a foreach)

frutas := []string{"maçã", "banana", "uva"}

for indice, fruta := range frutas {

    fmt.Printf("[%d] %s\n", indice, fruta)

}

// Ignorar índice com _ (blank identifier)

for _, fruta := range frutas {

    fmt.Println(fruta)

}

// SWITCH — não precisa de break (Go faz break implícito)

dia := "segunda"
```

```
switch dia {  
  
case "segunda", "terça", "quarta", "quinta", "sexta":
```

```
    fmt.Println("Dia útil")
```

```
case "sábado", "domingo":
```

```
    fmt.Println("Fim de semana")
```

```
default:
```

```
    fmt.Println("Dia inválido")
```

```
}
```

```
// Switch sem expressão (equivalente a if/else if encadeado — mais limpo)
```

```
nota := 7
```

```
switch {
```

```
case nota >= 9:
```

```
    fmt.Println("Excelente")
```

```
case nota >= 7:
```

```
    fmt.Println("Bom")
```

```
case nota >= 5:
```

```
    fmt.Println("Regular")
```

```
default:
```

```
    fmt.Println("Insuficiente")
```

```
}
```

```
}
```

Comparação com PHP: | Conceito | PHP | Go | |---|---|---| | if | `if ($x > 5) {}` | `if x > 5 {}` (sem parênteses) | | for | `for ($i=0; $i<5; $i++)` | `for i := 0; i < 5; i++` | | foreach | `foreach ($arr as $k => $v)` | `for k, v := range arr` | | while | `while ($x < 3)` | `for x < 3` | | switch break | Precisa de `break` | Break implícito |

1.5 — Funções

O que você aprende: funções em Go podem retornar múltiplos valores (recurso matador que elimina muitos patterns de erro). Não existem exceções — erros são retornados explicitamente como segundo valor.

```
package main
```

```
import (
```

```
    "errors"
```

```
    "fmt"
```

```
)
```

```
// Função simples
```

```
func saudacao(nome string) string {
```

```
    return fmt.Sprintf("Olá, %s!", nome)
```

```
}
```

```
// Múltiplos parâmetros do mesmo tipo (atalho)
```

```
func soma(a, b int) int {
```

```
    return a + b
```

```
}
```

```
// MÚLTIPLOS RETORNOS — o padrão mais importante do Go
```

```
// Quase toda função que pode falhar retorna (resultado, erro)
```

```

func dividir(a, b float64) (float64, error) {

    if b == 0 {

        return 0, errors.New("divisão por zero")

    }

    return a / b, nil // nil = sem erro

}

// Retornos nomeados (útil em funções complexas)

func estatisticas(numeros []int) (soma int, media float64) {

    for _, n := range numeros {

        soma += n

    }

    media = float64(soma) / float64(len(numeros))

    return // retorna soma e media automaticamente

}

// Função variádica (aceita N argumentos — como ...$args em PHP)

func somarTodos(numeros ...int) int {

    total := 0

    for _, n := range numeros {

        total += n

    }

    return total

}

```

```
// Funções como valores (primeira classe — como closures em PHP/Python)
```

```
func aplicar(numeros []int, fn func(int) int) []int {
```

```
    resultado := make([]int, len(numeros))
```

```
    for i, n := range numeros {
```

```
        resultado[i] = fn(n)
```

```
    }
```

```
    return resultado
```

```
}
```

```
func main() {
```

```
    fmt.Println(saudacao("Aerlon"))
```

```
    fmt.Println(soma(3, 7))
```

```
// Padrão de tratamento de erro do Go (vai ver isso MILHARES de vezes)
```

```
    resultado, err := dividir(10, 3)
```

```
    if err != nil {
```

```
        fmt.Println("Erro:", err)
```

```
        return
```

```
    }
```

```
    fmt.Printf("10 / 3 = %.2f\n", resultado)
```

```
// Tentando divisão por zero
```

```
    _, err = dividir(10, 0)
```

```
    if err != nil {
```

```

    fmt.Println("Erro capturado:", err)

}

// Variádica

fmt.Println(somarTodos(1, 2, 3, 4, 5)) // 15

// Função como valor

dobro := func(n int) int { return n * 2 }

nums := []int{1, 2, 3}

fmt.Println(aplicar(nums, dobro)) // [2 4 6]

}

```

O pattern `if err != nil` — aceite agora: Go não tem `try/catch`. Em vez disso, funções retornam um `error` como segundo valor. Você checa com `if err != nil` após cada chamada. Parece verboso no início, mas força tratamento de erro explícito em todo lugar — sem surpresas em produção.

1.6 — Structs e Métodos

O que você aprende: Go não tem classes. Em vez disso, usa **structs** (como structs em C, mas com métodos). É a base de toda modelagem de dados.

```

package main

import (
    "fmt"
    "time"
)

// Struct — define um tipo de dado composto (equivalente a uma classe sem herança)

type Tarefa struct {

```

```

    ID    int

    Titulo string

    Descricao string

    Feita    bool

    CriadaEm time.Time
}

// Método — função associada a um struct (receiver)

// (t Tarefa) = receiver por valor (cópia, não altera o original)

func (t Tarefa) Resumo() string {

    status := "Pendente"

    if t.Feita {

        status = "Feita"

    }

    return fmt.Sprintf("[%d] %s — %s", t.ID, t.Titulo, status)

}

// Método com pointer receiver — altera o struct original

// (t *Tarefa) = receiver por ponteiro (altera o original)

func (t *Tarefa) Concluir() {

    t.Feita = true

}

func (t *Tarefa) AlterarTitulo(novo string) {

```

```

    t.Titulo = novo
}

// Struct composto (composição em vez de herança)

type TarefaPrioritaria struct {

    Tarefa          // Embedding — "herda" todos os campos e métodos de Tarefa

    Prioridade string // campo adicional
}

func main() {

    // Criar instância (literal)

    t1 := Tarefa{

        ID:    1,

        Titulo: "Aprender Go",

        Descricao: "Estudar sintaxe completa do Go",

        Feita:    false,

        CriadaEm: time.Now(),

    }

    fmt.Println(t1.Resumo()) // [1] Aprender Go — Pendente

    // Chamar método que modifica (pointer receiver)

    t1.Concluir()

    fmt.Println(t1.Resumo()) // [1] Aprender Go — Feita

    // Struct com composição

    tp := TarefaPrioritaria{

```



```

Tarefa: Tarefa{

    ID:    2,

    Titulo: "Deploy API",

    CriadaEm: time.Now(),

},

Prioridade: "alta",

}

// Acessa métodos do Tarefa diretamente (embedding)

fmt.Println(tp.Resumo())    // [2] Deploy API — Pendente

fmt.Println(tp.Prioridade)  // alta

}

```

Conceito crítico — Pointer Receiver vs Value Receiver:

- `func (t Tarefa)` → recebe uma CÓPIA. Não modifica o original.
- `func (t *Tarefa)` → recebe um PONTEIRO. Modifica o original.
- **Regra prática:** se o método precisa alterar o struct, use `*`. Se só lê, pode usar valor. Na dúvida, use `*` sempre — é o mais comum.

1.7 — Ponteiros

O que você aprende: ponteiros são endereços de memória. Go usa ponteiros, mas de forma mais segura que C (sem aritmética de ponteiros). Você já viu ponteiros no pointer receiver — agora entende o mecanismo.

```
package main
```

```
import "fmt"
```

```
func main() {
```

```

// Variável normal

x := 42

// Ponteiro — armazena o ENDEREÇO de x

p := &x // & = "endereço de"

fmt.Println(x) // 42 (valor)

fmt.Println(p) // 0xc000018098 (endereço)

fmt.Println(*p) // 42 (desreferencia — pega o valor no endereço)

// Alterar via ponteiro

*p = 100

fmt.Println(x) // 100 — x mudou porque p aponta pra x

// Na prática: funções que precisam modificar o argumento

nome := "Aerlon"

adicionarSobrenome(&nome)

fmt.Println(nome) // "Aerlon Dev"

}

func adicionarSobrenome(s *string) {

    *s = *s + " Dev"

}

```

Quando usar ponteiro (regras práticas):

- Quando a função precisa **modificar** o argumento → ponteiro
- Quando o struct é **grande** e copiar é caro → ponteiro
- Quando pode ser **nil** (ausente) → ponteiro
- Caso contrário → valor (mais simples e seguro)

1.8 — Interfaces

O que você aprende: interfaces em Go são **implícitas** — não precisa declarar "implements". Se um struct tem os métodos da interface, ele satisfaz automaticamente. Isso é radicalmente diferente de PHP/Java.

```
package main
```

```
import "fmt"
```

```
// Interface — define um contrato (conjunto de métodos)
```

```
type Resumivel interface {
```

```
    Resumo() string
```

```
}
```

```
// Tarefa já implementa Resumivel (tem o método Resumo())
```

```
type Tarefa struct {
```

```
    ID    int
```

```
    Titulo string
```

```
    Feita bool
```

```
}
```

```
func (t Tarefa) Resumo() string {
```

```
    status := "Pendente"
```

```
    if t.Feita {
```

```
        status = "Feita"
```

```
    }
```

```

    return fmt.Sprintf("[%d] %s — %s", t.ID, t.Titulo, status)
}

// Projeto também implementa Resumível

type Projeto struct {

    Nome    string

    NumTarefas int
}

func (p Projeto) Resumo() string {

    return fmt.Sprintf("Projeto %s (%d tarefas)", p.Nome, p.NumTarefas)
}

// Função que aceita qualquer coisa que tenha Resumo()

func imprimir(item Resumível) {

    fmt.Println(item.Resumo())
}

// Interface vazia — aceita qualquer tipo (equivalente a mixed/any)

func descrever(i interface{}) {

    fmt.Printf("Tipo: %T | Valor: %v\n", i, i)
}

// A partir do Go 1.18, pode usar 'any' no lugar de 'interface{}'

func main() {

    t := Tarefa{ID: 1, Titulo: "Estudar interfaces", Feita: false}

    p := Projeto{Nome: "taskctl", NumTarefas: 5}

```

```

// Ambos satisfazem Resumivel — sem declarar "implements"

imprimir(t) // [1] Estudar interfaces — Pendente

imprimir(p) // Projeto taskctl (5 tarefas)

// Interface vazia aceita tudo

descrever(42)

descrever("texto")

descrever(t)

}

```

Diferença fundamental de PHP/Java: Em PHP: `class Tarefa implements Resumivel` — explícito. Em Go: se `Tarefa` tem `Resumo() string`, ela implementa `Resumivel` automaticamente. Sem declarar nada.

1.9 — Tratamento de Erros (o jeito Go)

O que você aprende: Go não tem exceções. Erros são valores retornados. É o aspecto que mais causa estranhamento pra quem vem de PHP/Python, mas depois de acostumar, você percebe que o código fica mais previsível.

```

package main

import (

    "errors"

    "fmt"

)

// Erros customizados (sentinelas)

var (

```

```

ErrTarefaNaoEncontrada = errors.New("tarefa não encontrada")

ErrTituloVazio      = errors.New("título não pode ser vazio")

)

// Erro com contexto usando fmt.Errorf

func buscarTarefa(id int, tarefas []string) (string, error) {

    if id < 0 || id >= len(tarefas) {

        return "", fmt.Errorf("id %d inválido: %w", id, ErrTarefaNaoEncontrada)

    }

    return tarefas[id], nil

}

func criarTarefa(titulo string) error {

    if titulo == "" {

        return ErrTituloVazio

    }

    fmt.Println("Tarefa criada:", titulo)

    return nil

}

func main() {

    tarefas := []string{"Estudar Go", "Fazer CRUD"}

    // Padrão: checar erro imediatamente após a chamada

    tarefa, err := buscarTarefa(0, tarefas)

    if err != nil {

```

```

    fmt.Println("Erro:", err)

    return
}

fmt.Println("Encontrada:", tarefa)

// Erro com verificação de tipo (errors.Is)

_, err = buscarTarefa(99, tarefas)

if err != nil {

    if errors.Is(err, ErrTarefaNaoEncontrada) {

        fmt.Println("Tarefa não existe!")

    } else {

        fmt.Println("Erro desconhecido:", err)

    }

}

// %w no fmt.Errorf permite encadear erros (wrapping)

// errors.Is() desce a cadeia pra achar o erro original

}

```

Conceitos importantes:

- `errors.New()` — cria erros simples (sentinelas)
- `fmt.Errorf("contexto: %w", err)` — wrapping (adiciona contexto mantendo o erro original)
- `errors.Is(err, alvo)` — verifica se um erro (mesmo wrapped) é de um tipo específico
- `errors.As(err, &target)` — converte pra um tipo de erro específico (menos comum)

1.10 — Packages e Organização de Projeto

O que você aprende: como organizar código em Go. Cada pasta é um package. Nomes exportados começam com letra maiúscula (público), minúscula = privado.

taskctl/

├── go.mod

├── main.go

├── model/

| └── tarefa.go

└── storage/

└── memory.go

model/tarefa.go:

```
package model
```

```
import "time"
```

```
// Tarefa é exportada (T maiúsculo) — acessível fora do package
```

```
type Tarefa struct {
```

```
    ID    int
```

```
    Titulo string
```

```
    Feita  bool
```

```
    CriadaEm time.Time
```

```
}
```

```
// validar é privada (v minúsculo) — só acessível dentro do package model
```



```
func validar(titulo string) bool {  
  
    return titulo != ""  
  
}  
  
// NovaTarefa é exportada — é o "construtor" (Go não tem construtores nativos)  
  
func NovaTarefa(id int, titulo string) *Tarefa {  
  
    return &Tarefa{  
  
        ID:    id,  
  
        Titulo: titulo,  
  
        Feita:  false,  
  
        CriadaEm: time.Now(),  
  
    }  
  
}
```

Regra de ouro — exportação por capitalização:

- `Tarefa` → público (exportado)
- `validar` → privado (não-exportado)
- Isso vale pra tudo: funções, structs, campos, constantes, variáveis.

Fase 2 — CRUD em Memória (API HTTP)

Objetivo: construir uma API REST funcional para o `taskctl` usando apenas a standard library do Go (sem frameworks). Ao final, você tem endpoints de Create, Read, Update, Delete operando em memória.

2.1 — Servidor HTTP Básico com `net/http`

O que você aprende: Go tem um servidor HTTP embutido na standard library — potente o suficiente pra produção. Não precisa de framework (diferente de PHP que precisa de Laravel, ou Python que precisa de Flask/FastAPI).

```
package main
```

```
import (
```

```
    "fmt"
```

```
    "net/http"
```

```
)
```

```
func main() {
```

```
    // Handler = função que responde a uma rota
```

```
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
```

```
        fmt.Fprintf(w, "taskctl API v0.1")
```

```
    })
```

```
    http.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
```

```
        w.Header().Set("Content-Type", "application/json")
```

```
        w.WriteHeader(http.StatusOK)
```

```
        fmt.Fprintf(w, `{"status": "ok"}`)
```

```
    })
```

```
    fmt.Println("Servidor rodando em :8080")
```

```
    // ListenAndServe bloqueia — o servidor fica escutando
```

```
    err := http.ListenAndServe(":8080", nil)
```

```

if err != nil {

    fmt.Println("Erro ao iniciar servidor:", err)

}

}

```

Conceitos novos:

- `http.HandleFunc(rota, handler)` — registra um handler pra uma rota
 - `http.ResponseWriter` — onde você escreve a resposta (equivalente a `response` do Laravel)
 - `*http.Request` — a requisição recebida (método, headers, body, query params)
 - `http.ListenAndServe` — inicia o servidor (bloqueante)
-

2.2 — Roteamento por Método HTTP

O que você aprende: `net/http` não diferencia GET/POST/PUT/DELETE automaticamente na rota. Você faz isso manualmente no handler. A partir do Go 1.22, o mux padrão suporta patterns com método.

```
package main
```

```
import (
```

```
    "encoding/json"
```

```
    "fmt"
```

```
    "net/http"
```

```
    "time"
```

```
)
```

```
type Tarefa struct {
```

```
    ID      int    `json:"id"`
```

```
    Titulo  string `json:"titulo"`
```

```

    Feita    bool    `json:"feita"`

    CriadaEm time.Time `json:"criada_em"`
}

// "Banco de dados" em memória

var tarefas = []Tarefa{}

var proximoID = 1

func main() {

    mux := http.NewServeMux()

    // Go 1.22+ suporta "METHOD /path" no pattern

    mux.HandleFunc("GET /tarefas", listarTarefas)

    mux.HandleFunc("POST /tarefas", criarTarefa)

    fmt.Println("API rodando em :8080")

    http.ListenAndServe(":8080", mux)
}

func listarTarefas(w http.ResponseWriter, r *http.Request) {

    w.Header().Set("Content-Type", "application/json")

    json.NewEncoder(w).Encode(tarefas)
}

func criarTarefa(w http.ResponseWriter, r *http.Request) {

    var input struct {

        Titulo string `json:"titulo"`
    }

```

```

}

// Decodificar JSON do body

err := json.NewDecoder(r.Body).Decode(&input)

if err != nil {

    http.Error(w, `{"erro": "JSON inválido"}`, http.StatusBadRequest)

    return

}

if input.Titulo == "" {

    http.Error(w, `{"erro": "titulo é obrigatório"}`, http.StatusBadRequest)

    return

}

tarefa := Tarefa{

    ID:    proximoID,

    Titulo: input.Titulo,

    Feita: false,

    CriadaEm: time.Now(),

}

proximoID++

tarefas = append(tarefas, tarefa)

w.Header().Set("Content-Type", "application/json")

w.WriteHeader(http.StatusCreated)

json.NewEncoder(w).Encode(tarefa)

```

```
}
```

Tags JSON (`json:"nome"`): As tags `json:"id"` nos campos do struct controlam como Go serializa/desserializa JSON. Sem tags, Go usa o nome do campo com primeira letra maiúscula (`ID`, `Titulo`). Com tags, você controla o nome exato no JSON.

2.3 — CRUD Completo (Update e Delete)

O que você aprende: completar os 4 endpoints. Lidar com path parameters manualmente (Go 1.22+ facilita com `{id}`).

```
package main
```

```
import (
```

```
    "encoding/json"
```

```
    "fmt"
```

```
    "net/http"
```

```
    "strconv"
```

```
    "time"
```

```
)
```

```
type Tarefa struct {
```

```
    ID      int    `json:"id"`
```

```
    Titulo  string `json:"titulo"`
```

```
    Feita   bool   `json:"feita"`
```

```
    CriadaEm time.Time `json:"criada_em"`
```

```
}
```

```
var tarefas = []Tarefa{
```

```

var proximoID = 1

func main() {

    mux := http.NewServeMux()

    mux.HandleFunc("GET /tarefas", listarTarefas)

    mux.HandleFunc("POST /tarefas", criarTarefa)

    mux.HandleFunc("GET /tarefas/{id}", buscarTarefa)

    mux.HandleFunc("PUT /tarefas/{id}", atualizarTarefa)

    mux.HandleFunc("DELETE /tarefas/{id}", deletarTarefa)

    fmt.Println("taskctl API rodando em :8080")

    http.ListenAndServe(":8080", mux)

}

// Helper — encontrar tarefa por ID e retornar o índice

func encontrarTarefa(id int) (int, *Tarefa) {

    for i := range tarefas {

        if tarefas[i].ID == id {

            return i, &tarefas[i]

        }

    }

    return -1, nil

}

// Helper — extrair ID do path parameter

```

```

func extrairID(r *http.Request) (int, error) {

    idStr := r.PathValue("id") // Go 1.22+

    return strconv.Atoi(idStr)

}

func listarTarefas(w http.ResponseWriter, r *http.Request) {

    w.Header().Set("Content-Type", "application/json")

    json.NewEncoder(w).Encode(tarefas)

}

func criarTarefa(w http.ResponseWriter, r *http.Request) {

    var input struct {

        Titulo string `json:"titulo"`

    }

    if err := json.NewDecoder(r.Body).Decode(&input); err != nil {

        http.Error(w, `{"erro":"JSON inválido"}`, http.StatusBadRequest)

        return

    }

    if input.Titulo == "" {

        http.Error(w, `{"erro":"titulo obrigatório"}`, http.StatusBadRequest)

        return

    }

    tarefa := Tarefa{

        ID:      proximoID,

```



```

        Titulo: input.Titulo,

        Feita: false,

        CriadaEm: time.Now(),
    }

    proximoID++

    tarefas = append(tarefas, tarefa)

    w.Header().Set("Content-Type", "application/json")

    w.WriteHeader(http.StatusCreated)

    json.NewEncoder(w).Encode(tarefa)
}

func buscarTarefa(w http.ResponseWriter, r *http.Request) {

    id, err := extrairID(r)

    if err != nil {

        http.Error(w, `{"erro":"id inválido"}`, http.StatusBadRequest)

        return

    }

    _, tarefa := encontrarTarefa(id)

    if tarefa == nil {

        http.Error(w, `{"erro":"tarefa não encontrada"}`, http.StatusNotFound)

        return

    }

```

```

w.Header().Set("Content-Type", "application/json")

json.NewEncoder(w).Encode(tarefa)
}

func atualizarTarefa(w http.ResponseWriter, r *http.Request) {

    id, err := extrairID(r)

    if err != nil {

        http.Error(w, `{"erro":"id inválido"}`, http.StatusBadRequest)

        return

    }

    _, tarefa := encontrarTarefa(id)

    if tarefa == nil {

        http.Error(w, `{"erro":"tarefa não encontrada"}`, http.StatusNotFound)

        return

    }

    var input struct {

        Titulo *string `json:"titulo"`

        Feita *bool `json:"feita"`

    }

    if err := json.NewDecoder(r.Body).Decode(&input); err != nil {

        http.Error(w, `{"erro":"JSON inválido"}`, http.StatusBadRequest)

        return

    }

```

```

// Ponteiros permitem distinguir "não enviou" (nil) de "enviou vazio"

if input.Titulo != nil {

    tarefa.Titulo = *input.Titulo

}

if input.Feita != nil {

    tarefa.Feita = *input.Feita

}

w.Header().Set("Content-Type", "application/json")

json.NewEncoder(w).Encode(tarefa)

}

func deletarTarefa(w http.ResponseWriter, r *http.Request) {

    id, err := extrairID(r)

    if err != nil {

        http.Error(w, `{"erro":"id inválido"}`, http.StatusBadRequest)

        return

    }

    idx, _ := encontrarTarefa(id)

    if idx == -1 {

        http.Error(w, `{"erro":"tarefa não encontrada"}`, http.StatusNotFound)

        return

    }

```

```
// Remove do slice (Go não tem "delete" pra slice — faz reslicing)

tarefas = append(tarefas[:idx], tarefas[idx+1:]...)

w.WriteHeader(http.StatusNoContent) // 204 — sem body

}
```

Padrão importante — ponteiros no input do Update: Usar `*string` e `*bool` no input permite distinguir entre "o campo não foi enviado" (`nil`) e "o campo foi enviado vazio/falso". Sem isso, `false` e "não enviou" seriam indistinguíveis (ambos seriam o zero value).

2.4 — Middleware

O que você aprende: middleware em Go é uma função que recebe um handler e retorna outro handler — wrapping simples, sem magia.

// Middleware de logging

```
func logMiddleware(next http.Handler) http.Handler {

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {

        start := time.Now()

        fmt.Printf("[%s] %s %s\n", time.Now().Format("15:04:05"), r.Method, r.URL.Path)

        next.ServeHTTP(w, r) // chama o próximo handler

        fmt.Printf("↳ concluído em %v\n", time.Since(start))

    })

}
```

// No main():

// Em vez de: `http.ListenAndServe(":8080", mux)`

// Usa: `http.ListenAndServe(":8080", logMiddleware(mux))`

Fase 3 — Banco de Dados e Primeiro Sistema Rodando

Objetivo: trocar o armazenamento em memória por PostgreSQL, usar migrations, e ter o `taskctl` como um sistema real que sobrevive a restarts.

3.1 — Conectando ao PostgreSQL

O que você aprende: como conectar Go ao banco de dados. Vamos usar `pgx` (driver mais popular e performático pra PostgreSQL em Go — mais moderno que o `database/sql` padrão).

Instalar o driver

```
go get github.com/jackc/pgx/v5
```

```
go get github.com/jackc/pgx/v5/pgxpool
```

Conceito importante — `pgxpool` vs `pgx`:

- `pgx.Conn` = uma conexão única (não use em APIs web — não é thread-safe)
- `pgxpool.Pool` = pool de conexões (use este — gerencia conexões automaticamente, thread-safe pra uso concorrente com goroutines)

```
// db/postgres.go
```

```
package db
```

```
import (
```

```
    "context"
```

```
    "fmt"
```

```
    "os"
```

```
    "github.com/jackc/pgx/v5/pgxpool"
```

```
)
```

```

func NovoPool() (*pgxpool.Pool, error) {

    // Pega a URL do banco de dados do ambiente

    dbURL := os.Getenv("DATABASE_URL")

    if dbURL == "" {

        dbURL = "postgres://postgres:postgres@localhost:5432/taskctl?sslmode=disable"

    }

    pool, err := pgxpool.New(context.Background(), dbURL)

    if err != nil {

        return nil, fmt.Errorf("erro ao conectar no banco: %w", err)

    }

    // Testar a conexão

    err = pool.Ping(context.Background())

    if err != nil {

        return nil, fmt.Errorf("banco não respondeu ao ping: %w", err)

    }

    fmt.Println("Conectado ao PostgreSQL")

    return pool, nil

}

```

Conceito novo — `context.Context`: Go usa `context` pra controlar timeout, cancelamento e propagação de valores entre goroutines. Quase toda função de I/O (banco, HTTP, arquivo) aceita um `context` como primeiro parâmetro. Por enquanto, use `context.Background()` (contexto vazio, sem timeout). Em APIs web, vai usar `r.Context()` (contexto da requisição HTTP).

3.2 — Criando a Tabela (Migration Manual)

O que você aprende: como criar o schema do banco. Em projetos reais, você usaria uma ferramenta de migration como `golang-migrate` ou `goose` — por agora, fazemos manual pra entender o fluxo.

```
-- migrations/001_criar_tarefas.sql

CREATE TABLE IF NOT EXISTS tarefas (

    id      SERIAL PRIMARY KEY,

    titulo  VARCHAR(255) NOT NULL,

    feita   BOOLEAN NOT NULL DEFAULT FALSE,

    criada_em TIMESTAMP NOT NULL DEFAULT NOW()

);

# Criar banco e rodar migration

createdb taskctl

psql taskctl < migrations/001_criar_tarefas.sql
```

3.3 — Repository Pattern (Camada de Acesso a Dados)

O que você aprende: separar o acesso ao banco da lógica HTTP. O repository encapsula todas as queries — se um dia trocar PostgreSQL por outro banco, só muda o repository.

```
// repository/tarefa_repo.go

package repository

import (

    "context"

    "fmt"
```

```

    "time"

    "github.com/jackc/pgx/v5/pgxpool"

)

type Tarefa struct {

    ID      int      `json:"id"`

    Titulo   string   `json:"titulo"`

    Feita    bool     `json:"feita"`

    CriadaEm time.Time `json:"criada_em"`

}

type TarefaRepository struct {

    pool *pgxpool.Pool

}

func NovoTarefaRepository(pool *pgxpool.Pool) *TarefaRepository {

    return &TarefaRepository{pool: pool}

}

func (r *TarefaRepository) Listar(ctx context.Context) ([]Tarefa, error) {

    rows, err := r.pool.Query(ctx,

        "SELECT id, titulo, feita, criada_em FROM tarefas ORDER BY id")

    if err != nil {

        return nil, fmt.Errorf("erro ao listar tarefas: %w", err)

    }

    defer rows.Close()

```



```

var tarefas []Tarefa

for rows.Next() {

    var t Tarefa

    err := rows.Scan(&t.ID, &t.Titulo, &t.Feita, &t.CriadaEm)

    if err != nil {

        return nil, fmt.Errorf("erro ao ler tarefa: %w", err)

    }

    tarefas = append(tarefas, t)

}

return tarefas, nil
}

func (r *TarefaRepository) BuscarPorID(ctx context.Context, id int) (*Tarefa, error) {

    var t Tarefa

    err := r.pool.QueryRow(ctx,

        "SELECT id, titulo, feita, criada_em FROM tarefas WHERE id = $1", id,

    ).Scan(&t.ID, &t.Titulo, &t.Feita, &t.CriadaEm)

    if err != nil {

        return nil, fmt.Errorf("tarefa %d não encontrada: %w", id, err)

    }

    return &t, nil

}

```

```

func (r *TarefaRepository) Criar(ctx context.Context, titulo string) (*Tarefa, error) {

    var t Tarefa

    err := r.pool.QueryRow(ctx,

        `INSERT INTO tarefas (titulo) VALUES ($1)

        RETURNING id, titulo, feita, criada_em`, titulo,

    ).Scan(&t.ID, &t.Titulo, &t.Feita, &t.CriadaEm)

    if err != nil {

        return nil, fmt.Errorf("erro ao criar tarefa: %w", err)

    }

    return &t, nil

}

func (r *TarefaRepository) Atualizar(ctx context.Context, id int, titulo *string, feita *bool) (*Tarefa, error) {

    // Busca atual, aplica mudanças, salva

    tarefa, err := r.BuscarPorID(ctx, id)

    if err != nil {

        return nil, err

    }

    if titulo != nil {

        tarefa.Titulo = *titulo

    }

    if feita != nil {

```

```

        tarefa.Feita = *feita
    }

    _, err = r.pool.Exec(ctx,

        "UPDATE tarefas SET titulo = $1, feita = $2 WHERE id = $3",

        tarefa.Titulo, tarefa.Feita, id)

    if err != nil {

        return nil, fmt.Errorf("erro ao atualizar tarefa: %w", err)

    }

    return tarefa, nil
}

func (r *TarefaRepository) Deletar(ctx context.Context, id int) error {

    result, err := r.pool.Exec(ctx, "DELETE FROM tarefas WHERE id = $1", id)

    if err != nil {

        return fmt.Errorf("erro ao deletar tarefa: %w", err)

    }

    if result.RowsAffected() == 0 {

        return fmt.Errorf("tarefa %d não encontrada", id)

    }

    return nil
}

```

Padrões importantes:

- `$1`, `$2`, `$3` — placeholders do PostgreSQL (previnem SQL injection — nunca concatene strings!)
 - `RETURNING` — retorna os campos inseridos sem precisar de `SELECT` extra
 - `defer rows.Close()` — garante que o cursor do banco é fechado mesmo se der erro
 - `rows.Next()` + `rows.Scan()` — itera sobre resultados (parecido com cursor de banco em Python)
-

3.4 — Integrando Repository com a API HTTP

O que você aprende: conectar tudo — HTTP handlers chamam o repository, que acessa o banco. Estrutura final do projeto.

taskctl/

├── go.mod

├── go.sum

├── main.go

├── db/

| └── postgres.go

├── repository/

| └── tarefa_repo.go

├── handler/

| └── tarefa_handler.go

└── migrations/

└── 001_criar_tarefas.sql

// handler/tarefa_handler.go

package handler

```

import (

    "encoding/json"

    "net/http"

    "strconv"

    "github.com/aerlon/taskctl/repository"
)

type TarefaHandler struct {

    repo *repository.TarefaRepository
}

func NovoTarefaHandler(repo *repository.TarefaRepository) *TarefaHandler {

    return &TarefaHandler{repo: repo}
}

func (h *TarefaHandler) Listar(w http.ResponseWriter, r *http.Request) {

    tarefas, err := h.repo.Listar(r.Context())

    if err != nil {

        http.Error(w, `{"erro": "falha ao listar"}`, http.StatusInternalServerError)

        return
    }

    w.Header().Set("Content-Type", "application/json")

    json.NewEncoder(w).Encode(tarefas)
}

func (h *TarefaHandler) Criar(w http.ResponseWriter, r *http.Request) {

```

```

var input struct {

    Titulo string `json:"titulo"`

}

if err := json.NewDecoder(r.Body).Decode(&input); err != nil {

    http.Error(w, `{"erro":"JSON inválido"}`, http.StatusBadRequest)

    return

}

if input.Titulo == "" {

    http.Error(w, `{"erro":"titulo obrigatório"}`, http.StatusBadRequest)

    return

}

tarefa, err := h.repo.Criar(r.Context(), input.Titulo)

if err != nil {

    http.Error(w, `{"erro":"falha ao criar"}`, http.StatusInternalServerError)

    return

}

w.Header().Set("Content-Type", "application/json")

w.WriteHeader(http.StatusCreated)

json.NewEncoder(w).Encode(tarefa)

}

func (h *TarefaHandler) Buscar(w http.ResponseWriter, r *http.Request) {

```

```

id, err := strconv.Atoi(r.PathValue("id"))

if err != nil {

    http.Error(w, `{"erro":"id inválido"}`, http.StatusBadRequest)

    return

}

tarefa, err := h.repo.BuscarPorID(r.Context(), id)

if err != nil {

    http.Error(w, `{"erro":"não encontrada"}`, http.StatusNotFound)

    return

}

w.Header().Set("Content-Type", "application/json")

json.NewEncoder(w).Encode(tarefa)

}

func (h *TarefaHandler) Atualizar(w http.ResponseWriter, r *http.Request) {

    id, err := strconv.Atoi(r.PathValue("id"))

    if err != nil {

        http.Error(w, `{"erro":"id inválido"}`, http.StatusBadRequest)

        return

    }

    var input struct {

        Titulo *string `json:"titulo"`

        Feita *bool `json:"feita"`

```

```

    }

    if err := json.NewDecoder(r.Body).Decode(&input); err != nil {

        http.Error(w, `{"erro":"JSON inválido"}`, http.StatusBadRequest)

        return

    }

    tarefa, err := h.repo.Atualizar(r.Context(), id, input.Titulo, input.Feita)

    if err != nil {

        http.Error(w, `{"erro":"falha ao atualizar"}`, http.StatusNotFound)

        return

    }

    w.Header().Set("Content-Type", "application/json")

    json.NewEncoder(w).Encode(tarefa)

}

func (h *TarefaHandler) Deletar(w http.ResponseWriter, r *http.Request) {

    id, err := strconv.Atoi(r.PathValue("id"))

    if err != nil {

        http.Error(w, `{"erro":"id inválido"}`, http.StatusBadRequest)

        return

    }

    if err := h.repo.Deletar(r.Context(), id); err != nil {

        http.Error(w, `{"erro":"falha ao deletar"}`, http.StatusNotFound)

```



```
        return

    }

    w.WriteHeader(http.StatusNoContent)

}

// main.go — tudo conectado

package main

import (

    "fmt"

    "log"

    "net/http"

    "github.com/aerlon/taskctl/db"

    "github.com/aerlon/taskctl/handler"

    "github.com/aerlon/taskctl/repository"

)

func main() {

    // 1. Conectar ao banco

    pool, err := db.NovoPool()

    if err != nil {

        log.Fatal(err)

    }

    defer pool.Close()

    // 2. Criar repository e handler
```

```
repo := repository.NovoTarefaRepository(pool)

h := handler.NovoTarefaHandler(repo)

// 3. Registrar rotas

mux := http.NewServeMux()

mux.HandleFunc("GET /tarefas", h.Listar)

mux.HandleFunc("POST /tarefas", h.Criar)

mux.HandleFunc("GET /tarefas/{id}", h.Buscar)

mux.HandleFunc("PUT /tarefas/{id}", h.Atualizar)

mux.HandleFunc("DELETE /tarefas/{id}", h.Deletar)

// 4. Subir servidor

fmt.Println("taskctl API rodando em :8080")

log.Fatal(http.ListenAndServe(":8080", mux))

}
```

Testando com curl:

Criar

```
curl -X POST http://localhost:8080/tarefas \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"titulo": "Aprender Go"}'
```

Listar

```
curl http://localhost:8080/tarefas
```

Buscar

```
curl http://localhost:8080/tarefas/1
```

Atualizar

```
curl -X PUT http://localhost:8080/tarefas/1 \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"feita": true}'
```

Deletar

```
curl -X DELETE http://localhost:8080/tarefas/1
```

Fases Seguintes (preview)

Posso detalhar cada uma dessas no mesmo nível das fases 1-3:

Fase	Tema	O que cobre
4	Goroutines e Concorrência	goroutines, channels, sync.WaitGroup, sync.Mutex, patterns de worker pool — o diferencial do Go
5	Testes	testing package, table-driven tests, mocks, httptest, benchmarks
6	Router e Frameworks	Chi ou Gin, middleware avançado, graceful shutdown, configuração com Viper
7	Docker e Deploy	Dockerfile multi-stage, docker-compose com PostgreSQL, variáveis de ambiente
8	gRPC e Protobuf	definição de serviços .proto, geração de código, servidor

Fase	Tema	O que cobre
		gRPC — diferencial das vagas
9	Kubernetes Básico	Pods, Deployments, Services, ConfigMaps, HPA — orquestração de containers
10	Projeto Final Completo	taskctl com autenticação JWT, rate limiting, observabilidade, CI/CD, pronto pra portfólio

Observação: este plano foi pensado pra quem já programa (PHP/Python) e precisa aprender Go rápido pra mercado. Os conceitos são explicados com comparações a linguagens que você já conhece. O sistema `taskctl` evolui junto com cada fase — no final, é um projeto real de portfólio.